

“Linux Inter-Process Communication Using Pipes and FIFOs”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

**SURAPURAJU HIMASREE
MALLULA LALITHA
SRAVYA PERI**

**2520090181
2520080063
2520090059**

Under the Esteemed Guidance of

Dasari Alekhya

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Linux Inter-Process Communication Using Pipes and FIFOs”** is a record of Bonafide work of **SURAPURAJU HIMASREE - 2520090181, MALLULA LALITHA - 2520080063, SRAVYA PERI - 2520090059** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

SURAPURAJU HIMASREE	2520090181
MALLULA LALITHA	2520080063
SRAVYA PERI	2520090059

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled **“Linux Inter-Process Communication Using Pipes and FIFOs”** is a bonafide work done and submitted by **SURAPURAJU HIMASREE - 2520090181, MALLULA LALITHA - 2520080063, SRAVYA PERI - 2520090059,**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our guide **Dasari Alekhya**, Department of Computer Science and Engineering, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

SURAPURAJU HIMASREE **2520090181**

MALLULA LALITHA **2520080063**

SRAVYA PERI **2520090059**

Abstract

Inter-Process Communication (IPC) is an important concept in operating systems that enables multiple processes to exchange information and coordinate their activities. This project, "Linux Inter-Process Communication Using Pipes and FIFOs," focuses on understanding and implementing two fundamental IPC mechanisms available in Linux: anonymous pipes and named pipes (FIFOs). Pipes provide a unidirectional communication channel that is commonly used between related processes, such as a parent and child process. FIFOs extend this concept by allowing communication between unrelated processes through a special file created in the file system.

The project demonstrates the complete process of communication, including creating communication channels, sending data from one process to another, reading the transmitted data, and properly closing the communication resources. Linux system calls such as `pipe()`, `mkfifo()`, `fork()`, `read()`, `write()`, and `close()` are used to implement these mechanisms. The project also demonstrates synchronization between processes to ensure that data is transferred correctly without conflicts or loss.

Through this implementation, the project provides practical knowledge of process creation, inter-process data exchange, synchronization, and file-based communication in a Linux environment. The performance and behaviour of pipes and FIFOs can also be observed under different communication scenarios. Overall, this project helps develop a strong understanding of how Linux manages communication between concurrent processes and demonstrates how IPC mechanisms can be used to build efficient and coordinated multi-process applications. It serves as a practical foundation for studying advanced Linux operating-system concepts such as process synchronization, concurrent programming, and communication mechanisms.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8–11
		1.1 Background of the Project	8
		1.2 Problem Statement	8
		1.3 Scope of the Project	9
		1.4 Objectives of the Project	9
		1.5 Importance of the Application	10
		1.6 Target Users / Audience	10
2	OS Concepts and Background Study		12–15
		2.1 Operating System Concepts Related to the Project	12
		2.2 Existing Methods / Algorithms	14
		2.3 Proposed Approach	15
		2.4 Advantages of the Proposed Approach	15
3	System Requirements		16–17
		3.1 Hardware Requirements	16
		3.2 Software Requirements	16
		3.3 Operating System Environment – Ubuntu / WSL	16
		3.4 Development Tools and Compilers	17
4	Technology and Tools Used		18–20
		4.1 C / C++ Programming	18
		4.2 GCC Compiler	18
		4.3 Linux System Calls	18
		4.4 POSIX Threads / Synchronization	19
		4.5 Linux Commands and Utilities	19
		4.6 Other Tools / Libraries Used	19
5	System Architecture and Design		21–26
		5.1 Overall System Architecture	21
		5.2 System Workflow	22
		5.3 Flowchart / Data Flow	24

		5.4 Module Description	25
		5.5 Input and Output Design	26
6	Implementation		27–38
		6.1 Module-wise Implementation	27
		6.2 Algorithm / Pseudocode	28
		6.3 Core C / C++ Implementation	28
		6.4 Linux System Calls / APIs Used	35
		6.5 Process / Thread Implementation	35
		6.6 IPC / Synchronization Implementation	36
		6.7 Memory Management Implementation	37
		6.8 Optional User Interface Implementation	38
7	Execution and Demonstration		39–47
		7.1 Compilation in Ubuntu	39
		7.2 Execution Commands	39
		7.3 Sample Inputs and Outputs	40
		7.4 Screenshots of Execution	40
		7.5 Demonstration of OS Concepts	47
8	Conclusion		48–50
		8.1 Summary of the Project	48
		8.2 Objectives Achieved	48
		8.3 Results Achieved	49
		8.4 Skills and Learning Outcomes	50
9	References		51–52
		9.1 Books	51
		9.2 Research Papers	51
		9.3 Linux / GNU Documentation	51
		9.4 Online Resources	51

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

A modern operating system runs many programs at the same time. Linux gives every running program, called a **process**, its own private virtual address space, so that one process cannot read or damage the memory of another. This isolation is the foundation of security and stability, but it creates a practical difficulty: cooperating programs still have to exchange data and coordinate their work. The operating system therefore provides **Inter-Process Communication (IPC)** mechanisms, which are controlled services offered by the kernel through system calls.

Linux supports many IPC mechanisms, including pipes, named pipes (FIFOs), message queues, shared memory, semaphores, sockets and signals. Pipes are the oldest and simplest of them and are used every time a user types a shell command such as `ls | grep txt`. An **anonymous pipe** is a unidirectional byte channel that exists only as a pair of file descriptors, so it can be shared only between related processes such as a parent and its child. A **named pipe (FIFO)** extends the same idea by giving the channel a name in the file system, so that completely unrelated processes can communicate through it.

This project, "Linux Inter-Process Communication Using Pipes and FIFOs", was carried out for the course Operating Systems and Systems Programming (25CS2104E), 2026-27, Term-I. It implements both mechanisms in the C programming language using Linux system calls such as `pipe()`, `mkfifo()`, `fork()`, `read()`, `write()` and `close()`. The implementation is delivered as one integrated console application, the **Linux IPC Monitoring and Message Transfer System**, in which real messages typed by the user travel from a sender process to a receiver process while the program shows every system call, process identifier and file descriptor involved.

1.2 Problem Statement

In a Linux operating system, multiple processes often need to exchange data and coordinate their execution. Since processes have separate memory spaces, they cannot directly access each other's data. This creates a need for reliable Inter-Process Communication mechanisms. Without proper communication and synchronization, processes may experience data loss, incorrect data transfer or synchronization problems.

This project addresses the problem of establishing reliable communication between processes using pipes and FIFOs. Anonymous pipes are used for communication between related

processes, while named pipes enable communication between unrelated processes. The system demonstrates how processes transfer data, synchronize their operations and manage communication resources using `pipe()`, `mkfifo()`, `fork()`, `read()` and `write()`.

The problem is directly relevant to Operating Systems because process communication, synchronization and resource management are fundamental responsibilities of the OS. It is relevant to Systems Programming because the solution is built on low-level Linux system calls that interact directly with processes and communication resources.

1.3 Scope of the Project

The scope of the project is limited to IPC between processes running on a single Linux machine. The work covers the following areas:

- Creation and use of **anonymous pipes** for parent-child communication.
- Creation and use of **named pipes (FIFOs)** for communication between separate reader and writer processes.
- Process creation and control with `fork()` and `waitpid()`, including process identifiers (PID and PPID).
- Synchronization between processes using signals (`SIGUSR1`, `SIGINT`), blocking `open()` semantics and end-of-file detection.
- Processing of received messages by worker threads that share a mutex-protected message queue and statistics.
- Observation of memory behaviour (heap buffers and Copy-on-Write after `fork()`) and file-system behaviour (inode, file type and permissions of a FIFO).

The following are outside the scope of the project: network IPC through sockets, System V or POSIX shared memory, message queues and semaphores, communication between different machines, and any graphical user interface. The application is an educational demonstrator and not a production messaging system.

1.4 Objectives of the Project

The main objectives of the project, as stated in the approved problem statement, are:

1. To implement inter-process communication between related processes using anonymous pipes in Linux.
2. To implement communication between unrelated processes using named pipes (FIFOs).
3. To demonstrate reliable data transfer, synchronization and coordination between

processes using Linux system calls.

4. To understand and apply Linux IPC concepts and system calls such as `pipe()`, `mkfifo()`, `fork()`, `read()`, `write()` and `close()`.

To achieve these objectives the implementation additionally aims to:

- transfer real, user-entered messages between processes instead of fixed demonstration strings;
- show every system call together with its visible return value, such as file descriptor numbers and child PIDs;
- visualize memory management and Copy-on-Write behaviour after `fork()`;
- connect the FIFO implementation to file-system concepts such as inode, file type, permissions and file descriptors;
- demonstrate a thread race condition and its removal with a mutex; and
- provide controlled signal handling and clean release of all resources (descriptors, heap memory, child processes and the FIFO file).

1.5 Importance of the Application

- **Practical learning:** operating-system topics such as processes, IPC, memory and file systems are usually taught theoretically. The application lets a learner observe each of them working inside one real program.
- **Foundation for advanced topics:** pipes and FIFOs lead naturally to process synchronization, concurrent programming and other IPC mechanisms.
- **Transparency:** all system calls are printed with their results, so the application can also be verified externally with tools such as `strace`, `ls -l` and `stat`.
- **Reliability practices:** every system call is checked for errors, descriptors are closed, memory is freed, children are reaped and the FIFO is removed, which are habits that every systems programmer needs.
- **Reusable design:** the sender, receiver, message queue and worker-thread modules are separate and can be reused in other multi-process programs.

1.6 Target Users / Audience

Table 1.1: Target users of the application

User group	How the project helps
B.Tech students of Operating Systems	Provides a working reference for pipes, FIFOs, fork, signals and threads that can be compiled, run and modified.

User group	How the project helps
Faculty and evaluators	Offers a menu-driven demonstration in which each concept can be shown separately or through one end-to-end run.
Beginners in Linux systems programming	Shows the correct order of system calls and the cleanup that is required after them.
Developers building multi-process tools	Serves as a small template for sender/receiver design using pipes or FIFOs.

CHAPTER -2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Process and address space

A process is a program in execution. The kernel keeps, for every process, a process identifier (PID), the identifier of its parent (PPID), a virtual address space made of text, data, heap and stack regions, and a table of open file descriptors. Because each process has a separate address space, a variable changed in one process is not visible in another. Communication therefore has to go through the kernel.

User space, kernel space and system calls

Application code runs in user space with restricted privileges. Services that need privileged access, such as creating a process or moving data between processes, are requested through **system calls**, which are the controlled entry points into the kernel. In this project every important operation, namely `pipe()`, `mkfifo()`, `fork()`, `read()`, `write()`, `close()`, `waitpid()`, `kill()` and `stat()`, is a system call (or a thin library wrapper around one), and the program prints the result of each so that the user/kernel boundary becomes visible.

Process creation and control

`fork()` creates a new process that is a copy of the calling process. It returns the child PID to the parent and 0 to the child, which is how the two copies decide which role to play. The parent uses `waitpid()` to wait for the child and collect its exit status; without this the terminated child would remain a **zombie** entry in the process table.

Anonymous pipes

`pipe()` creates a kernel buffer and returns two file descriptors: `pipefd[0]` for reading and `pipefd[1]` for writing. Data written to the write end can be read from the read end in first-in-first-out order. Since the descriptors are inherited through `fork()`, the parent and child can use them, and each process should close the end it does not need. When all write ends are closed, `read()` returns 0, which signals end-of-file to the reader.

Named pipes (FIFOs)

A FIFO behaves like a pipe but has a path name. It is created with `mkfifo()` and appears in the directory tree as a special file whose type is FIFO. Any process that has permission can open it, so unrelated processes can communicate. Opening a FIFO for reading blocks until another process opens it for writing, and the reverse also holds, which gives a natural rendezvous

between the two sides. The data itself is not stored on disk; it passes through a kernel buffer, and the file entry is removed with `unlink()`.

Signals

A signal is a small asynchronous notification sent to a process. The project uses `SIGUSR1` as a "receiver is ready" notification from child to parent and handles `SIGINT` for a controlled shutdown. Handlers are installed with `sigaction()` and are written to be async-signal-safe: they set a flag and use `write()` rather than `printf()`.

File descriptors, inodes and file types

A file descriptor is a small integer that indexes a process's open-file table. Descriptors 0, 1 and 2 are standard input, output and error, so the first descriptors returned by `pipe()` or `open()` in this program are 3 and 4. A file-system object is described by an **inode** that stores its type, permissions and size. `stat()` exposes this information and shows that a FIFO has an inode and the file type FIFO, whereas an anonymous pipe has no path name at all.

Threads, race conditions and mutual exclusion

Threads share the address space of their process. If several threads update the same variable without protection, the result depends on timing, which is a **race condition**. A **mutex** provides mutual exclusion: only the thread that holds the lock may execute the critical section. The project uses POSIX threads to process received messages in parallel and a mutex to protect the shared message queue and the statistics counters.

Memory management and Copy-on-Write

Message buffers are allocated on the heap with `malloc()` and released with `free()`. After `fork()` the parent and child initially share the same physical pages; the kernel copies a page only when one of the processes writes to it. This technique is called **Copy-on-Write (CoW)** and is demonstrated by changing a variable in the child and showing that the parent's copy is unchanged.

Table 2.1: OS concepts and their use in the project

OS concept	Linux API / mechanism	Where it appears in the project
System-call interface	pipe, fork, read, write, open, close	Every module; results printed on screen
Process management	fork, getpid, getppid, waitpid, _exit	Sender/receiver processes in <code>ipc_session.c</code>
IPC - anonymous pipe	pipe, read, write, close	Option 1 (anonymous pipe session)
IPC - named pipe	mkfifo, open, read, write, unlink	Option 2 (named FIFO session)
Signals	sigaction, kill, sigprocmask, sigsuspend	Ready notification and clean shutdown

OS concept	Linux API / mechanism	Where it appears in the project
File system	stat, inode, file descriptors	FIFO metadata display, option 4
Memory management	malloc, free, Copy-on-Write	Message buffers, option 5
Concurrency	pthread_create/join, pthread_mutex_*	Worker threads, option 6

2.2 Existing Methods / Algorithms

Linux offers several IPC methods, and each is suited to a different situation. Table 2.2 compares the commonly used ones with the two mechanisms chosen for this project.

Table 2.2: Comparison of IPC methods

Method	Related processes only?	Characteristics
Anonymous pipe	Yes	Unidirectional byte stream; no name; simple; ends with EOF when writers close.
Named pipe (FIFO)	No	Same as a pipe but has a file-system name; open() blocks until both ends are present.
Message queue	No	Discrete messages with types or priorities; kernel-persistent until removed.
Shared memory	No	Fastest, but the programmer must add synchronization (semaphores or mutexes).
Sockets	No	Bidirectional; works on one machine or across a network; more complex to set up.
Signals	No	Only a notification; carries almost no data; useful for events.
Ordinary files	No	Persistent but slow; needs polling and locking to coordinate.

In teaching, these concepts are usually shown as separate small programs: one for `fork()`, one for a pipe, one for a FIFO, and so on. Such programs often send a fixed string, hide the system-call results and do not show how the pieces work together. A shell pipeline such as `cat file | wc -l` uses the same mechanism but hides every system call from the learner. These existing approaches are correct but leave the learner without an end-to-end view.

Classical algorithm: producer-consumer through a bounded channel

A pipe or FIFO implements the classical producer-consumer pattern. The producer (writer) places bytes into a kernel buffer of limited size; if the buffer is full the producer blocks. The consumer (reader) removes bytes in the same order; if the buffer is empty the consumer blocks. The kernel therefore performs the synchronization, and the programmer only has to open, close and check return values correctly.

2.3 Proposed Approach

The project is designed and implemented in a Linux environment using the C programming language and Linux system calls. It demonstrates two IPC mechanisms, anonymous pipes and named pipes (FIFOs), inside one connected application.

Anonymous pipe mode. An anonymous pipe is created with `pipe()`. The parent process creates a child process with `fork()`. The processes communicate by writing data into the pipe and reading it from the other end. Unused file descriptors are closed in each process to ensure proper synchronization and communication.

Named pipe mode. A named pipe is created with `mkfifo()`. Separate processes use the FIFO as the communication channel: an independent writer process sends data and a reader process receives it. The `open()`, `read()`, `write()` and `close()` system calls are used for data transfer and resource management.

Overall flow. Both modes follow the sequence: Process Creation -> IPC Channel Creation -> Data Writing -> Data Reading -> Process Synchronization -> Resource Closure. The output demonstrates successful communication and data transfer between the processes. In addition, the receiver hands every message to a pool of three worker threads, and the program reports session statistics when the transfer is complete.

2.4 Advantages of the Proposed Approach

- **Integrated:** one application connects process creation, IPC, signals, memory, file-system and thread concepts through a single message-transfer workflow.
- **Real operations:** the messages are typed by the user and actually travel through a kernel pipe or FIFO; nothing is simulated.
- **Observable:** PIDs, descriptor numbers, byte counts, inode numbers and statistics are printed, and the program can be traced independently with `strace`.
- **Safe and clean:** all system calls are checked, signal handlers are async-signal-safe, descriptors are closed, memory is freed, children are reaped and the FIFO is unlinked.
- **Portable:** only POSIX system calls and the C standard library are used, with no external frameworks, so the program builds on Ubuntu and WSL with `gcc` and `make`.
- **Modular:** each concept lives in its own source file, which makes the code easy to read, test and extend.

CHAPTER -3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The application is small and has modest hardware needs. Table 3.1 lists the minimum and recommended configuration for compiling and running it, including running it inside Windows Subsystem for Linux (WSL).

Table 3.1: Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core 64-bit (x86-64) CPU	Quad-core 64-bit CPU (to observe real thread parallelism)
Memory (RAM)	2 GB	4 GB or more (8 GB when using WSL2 with an editor)
Storage	1 GB free (toolchain and sources)	5 GB free
Display and input	Terminal-capable display, keyboard	Two terminal windows to inspect the FIFO while a session runs

3.2 Software Requirements

Table 3.2: Software requirements

Software	Version / details	Purpose
Operating system	Ubuntu Linux (native, virtual machine or WSL)	Environment for building and running the IPC system
GCC	GNU C compiler (C99/C11 support)	Compiles and links the source code
GNU Make	Any recent version	Automates the build using the project Makefile
POSIX threads	Provided by glibc (-pthread)	Worker threads and mutexes
strace	Optional	Verifies the system calls made by the program
Valgrind	Optional	Checks for memory leaks and invalid memory use
Git	Optional	Version control; source is kept on GitHub
Text editor / IDE	Vim, nano or VS Code	Editing the source files

3.3 Operating System Environment - Ubuntu / WSL

The project targets Linux and is developed and tested in an Ubuntu environment. Users of Windows can run exactly the same code in WSL (Windows Subsystem for Linux) with an Ubuntu distribution, because WSL provides a real Linux system-call interface. The toolchain is installed with:

```
sudo apt update
sudo apt install gcc make
# optional verification tools
```

```
| sudo apt install strace valgrind git
```

Two practical points apply to the environment. First, the FIFO is created at `/tmp/os_project_fifo`, which lies on the Linux file system, so it works in native Ubuntu and in WSL without any special configuration. Second, the program is interactive and must be started from a terminal, because the sender process reads the messages that the user types.

3.4 Development Tools and Compilers

The code is compiled with GCC through a Makefile. The compiler is invoked with warnings enabled and with thread support, so that coding mistakes are reported at build time.

Table 3.3: Development tools

Tool	Role in development
<code>gcc -Wall -Wextra -pthread -linclude</code>	Compiles each <code>.c</code> file into an object file with all common warnings enabled
<code>make / make clean / make run</code>	Builds the program into <code>build/os_project</code> , removes the build directory, or builds and runs
<code>strace</code>	Traces pipe, fork, clone, read, write, openat, close, mkfifo and wait4 calls
<code>valgrind --leak-check=full</code>	Confirms that heap memory is released
<code>ls -l, stat</code>	Shows the FIFO entry, its type and inode while the program runs

CHAPTER -4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The project is written in the **C programming language** (C99/C11). C was chosen because Linux system calls are defined as C interfaces, and because C gives direct control over memory, pointers and file descriptors, which are exactly the resources this project studies. The code base contains about 2,000 lines spread across ten source files and nine header files. The features of C that are used most are structures (for the message queue and statistics), dynamic memory (`malloc`, `realloc`, `free`), pointers to pointers (an array of message strings), function pointers for thread entry points, and the preprocessor for shared constants such as `FIFO_PATH` and `BUFFER_SIZE`.

4.2 GCC Compiler

The GNU Compiler Collection (GCC) translates each `.c` file into an object file and then links the objects into the executable `build/os_project`. The flags used by the Makefile are explained in Table 4.1.

Table 4.1: GCC options used in the Makefile

Flag	Meaning
-Wall -Wextra	Enable the common and extra compiler warnings
-pthread	Compile and link with POSIX thread support
-Iinclude	Search the include/ directory for the project headers
-c file.c -o build/file.o	Compile one source file into an object file
gcc objects -o build/os_project	Link all object files into the final executable

4.3 Linux System Calls

System calls are the main technology of the project. Table 4.2 lists the calls used, the header that declares them, and their role.

Table 4.2: Linux system calls used

System call	Header	Role in the project
<code>pipe()</code>	<code>unistd.h</code>	Creates the anonymous pipe and returns two descriptors
<code>mkfifo()</code>	<code>sys/stat.h</code>	Creates the named pipe as a special file
<code>fork()</code>	<code>unistd.h</code>	Creates the receiver (child) process
<code>open()</code> / <code>close()</code>	<code>fcntl.h</code> / <code>unistd.h</code>	Opens and closes the FIFO and releases descriptors
<code>read()</code> / <code>write()</code>	<code>unistd.h</code>	Receives and sends the message bytes
<code>waitpid()</code>	<code>sys/wait.h</code>	Parent waits for the child and collects its status
<code>getpid()</code> / <code>getppid()</code>	<code>unistd.h</code>	Reports process and parent process identifiers

System call	Header	Role in the project
kill()	signal.h	Child sends SIGUSR1 to the parent
sigaction()	signal.h	Installs the SIGUSR1 and SIGINT handlers
sigprocmask() / sigsuspend()	signal.h	Blocks SIGUSR1 during FIFO open and waits for it safely
stat()	sys/stat.h	Reads the inode, type and permissions of the FIFO
unlink()	unistd.h	Removes the FIFO entry from the file system
_exit()	unistd.h	Terminates the child without running parent cleanup

4.4 POSIX Threads / Synchronization

POSIX threads (pthreads) provide the concurrency used after messages have been received. The functions used are `pthread_create()` to start a worker, `pthread_join()` to wait for it, `pthread_mutex_init()` and `pthread_mutex_destroy()` to manage a mutex, and `pthread_mutex_lock()` and `pthread_mutex_unlock()` to enter and leave a critical section. Two shared structures are protected by mutexes: the **message queue** (`MessageQueue`) and the **statistics counters** (`ipc_stats_t`). A separate diagnostic with four threads and a shared counter compares an unprotected update with a mutex-protected one.

4.5 Linux Commands and Utilities

Table 4.3: Commands and utilities

Command	Use during the project
make, make clean	Build the program and remove build output
./build/os_project	Run the menu-driven application
ls -l /tmp/os_project_fifo	Show the FIFO; the file type character is p
stat /tmp/os_project_fifo	Display inode number, type and permissions
strace -f -e trace=pipe,pipe2,fork,clone,read,write,openat,close ,mkfifo,wait4 ./build/os_project	Confirm the real system calls
ps aux grep Z	Check that no zombie processes remain
valgrind --leak-check=full ./build/os_project	Check for memory leaks

4.6 Other Tools / Libraries Used

- **C standard library:** `stdio.h`, `stdlib.h`, `string.h`, `errno.h` for input/output, memory, strings and error codes.
- **POSIX headers:** `unistd.h`, `fcntl.h`, `signal.h`, `sys/types.h`, `sys/stat.h`, `sys/wait.h` and `pthread.h`.
- **GNU Make:** a Makefile builds the ten source files and produces `build/os_project`.

- **Git and GitHub:** the source code is maintained in the GitHub repository <https://github.com/lalitha-0890/OS-Project>.
- **No external frameworks:** the project depends only on the C library and the Linux kernel interface.

CHAPTER -5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The application is organised in two layers. The upper layer is a user-space C program whose menu (`main.c`) calls the integrated IPC session module (`ipc_session.c`), the worker-thread module (`ipc_threads.c`) and four supporting diagnostic modules. The lower layer is the Linux kernel, which is reached only through the system-call interface and which provides process management, pipe and FIFO buffers, the virtual file system, memory management and signal delivery.

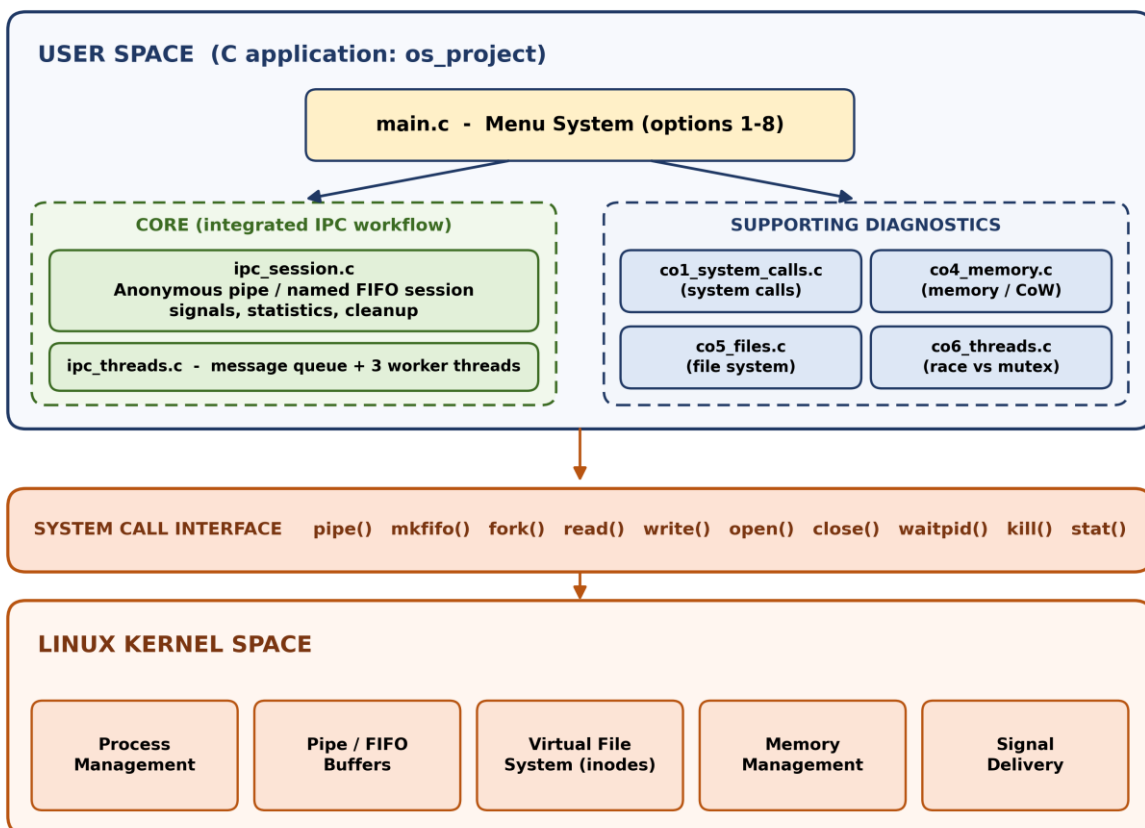


Figure 5.1: Overall system architecture

The core module `ipc_session.c` implements the real message-transfer workflow. The supporting modules `co1_system_calls.c`, `co4_memory.c`, `co5_files.c` and `co6_threads.c` provide focused diagnostics that can be run from the menu. The directory structure of the project is:

```
OS_Project/  
  Makefile  
  README.md  
  include/  common.h  ipc_session.h  ipc_threads.h  co1..co6 headers  
  src/      main.c    common.c  ipc_session.c  ipc_threads.c  
            co1_system_calls.c  co2_process_control.c  co3_ipc.c
```

	co4_memory.c	co5_files.c	co6_threads.c
docs/	architecture.md	system_calls.md	demonstration.md
tests/	test_script.sh		
build/	object files and the os_project executable		

5.2 System Workflow

Every IPC session, whichever mechanism is chosen, follows the same six-step workflow: Process Creation, IPC Channel Creation, Data Writing, Data Reading, Process Synchronization and Resource Closure.

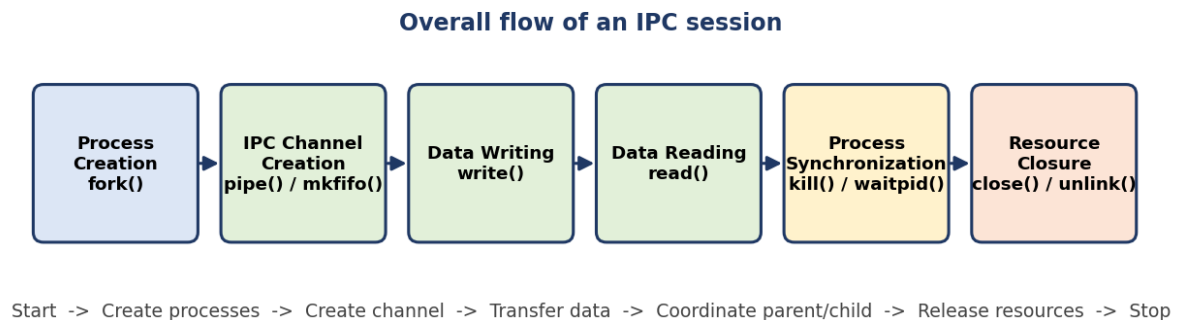
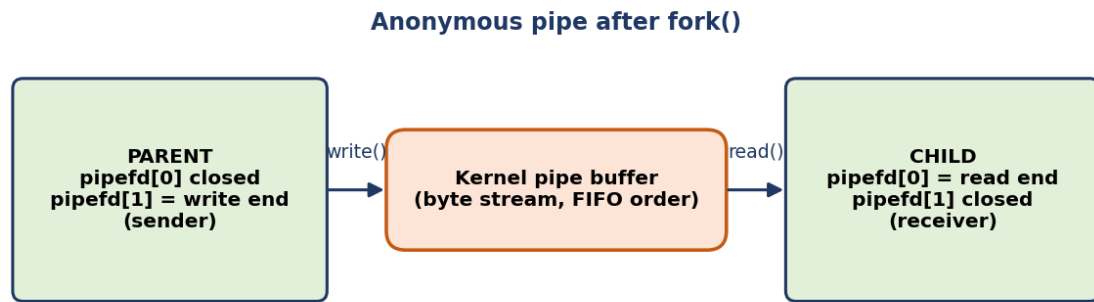


Figure 5.2: Workflow of an IPC session

Anonymous pipe session

1. The parent installs the signal handlers and creates the message-statistics mutex.
2. `pipe()` creates the data pipe, and a second pipe is created to carry the receiver's final statistics back to the parent.
3. `fork()` creates the child. The parent becomes the sender and the child becomes the receiver.
4. Each process closes the pipe end it does not use: the parent closes the read end and the child closes the write end.
5. The child sends `SIGUSR1` to tell the parent it is ready, and the parent waits for this signal.
6. The parent reads each message from the keyboard into a `malloc()` buffer and sends it with `write()`, followed by a newline delimiter.
7. The child reads the pipe byte by byte, splits the stream at newline characters and stores each message in the message queue.
8. When the user ends input (Ctrl+D), the parent closes the write end; the child sees end-of-file, starts three worker threads, prints the processing statistics and exits.
9. The parent calls `waitpid()`, reads the child's report from the statistics pipe and prints the session statistics.



Each process closes the end it does not use, so the reader sees EOF when the writer closes.

Figure 5.3: Descriptor layout of an anonymous pipe after fork()

Named FIFO session

1. `mkfifo("/tmp/os_project_fifo", 0666)` creates the FIFO, and `stat()` reports its file type, inode number and permissions.
2. The parent blocks `SIGUSR1` with `sigprocmask()` before forking, so that the signal cannot interrupt `open()`.
3. `fork()` creates the reader (child) and the writer (parent).
4. The child opens the FIFO with `O_RDONLY` and the parent opens it with `O_WRONLY`. Each `open()` blocks until the other side arrives.
5. After its open succeeds, the child sends `SIGUSR1`; the parent waits with `sigsuspend()` and then accepts messages from the user.
6. Each message is sent with `write()`; the child receives it with `read()` and queues it for the worker threads.
7. The parent closes the descriptor, the child sees end-of-file, the workers finish, the parent calls `waitpid()` and finally `unlink()` removes the FIFO.

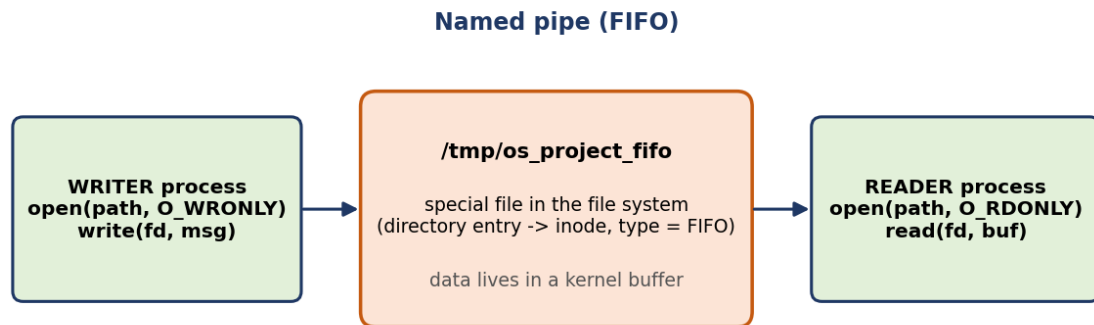


Figure 5.4: Communication through a named pipe (FIFO)

5.3 Flowchart / Data Flow

Figure 5.5 follows one message through the system. The text typed by the user is stored in a heap buffer, written by the sender into the kernel buffer of the pipe or FIFO, read by the receiver, placed in the thread-safe message queue and finally processed by one of the three worker threads, which update the shared statistics under a mutex.

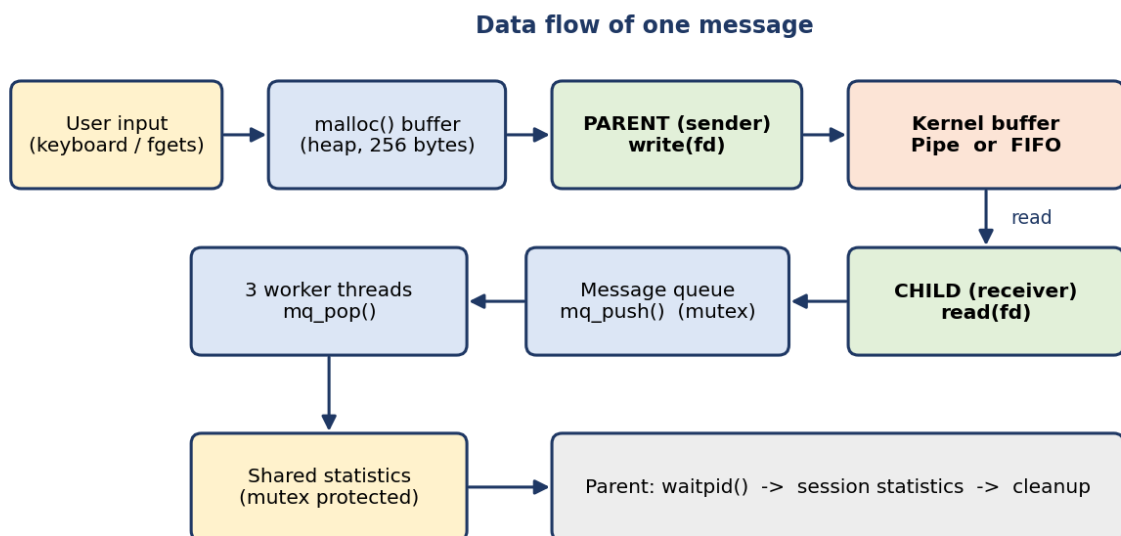


Figure 5.5: Data flow of one message

Statistics flow

Statistics are kept at two levels. The **session statistics** (messages sent, messages received, bytes transferred and errors) are updated by `stats_record_sent()`, `stats_record_received()` and `stats_record_error()` under a mutex. The **processing statistics** (messages received, messages processed and bytes processed) are updated by the worker threads through

`ipc_stats_record()`. Because the sender and receiver are separate processes with separate memory, the receiver in pipe mode returns its received-message count to the parent through the second pipe, which is itself an example of IPC.

Control flow of a session

Control flow

```
START
-> show menu  -> user selects option 1 (pipe) or 2 (FIFO)
-> create channel (pipe() or mkfifo())
-> fork()
    parent: sender                                child: receiver
    close unused end                               close unused end
    wait for SIGUSR1 <----- kill(parent, SIGUSR1)
    loop: read user message                         loop: read() until EOF
        write() to channel                           queue each message
    close write end ----- EOF -----> run 3 worker threads
    waitpid(child) <----- print statistics, _exit(0)
-> print session statistics, release resources
STOP
```

5.4 Module Description

Table 5.1: Modules and their responsibilities

Module (file)	Responsibility
main.c	Displays the eight-option menu, reads the choice and calls the selected module; exits safely on option 8 or end of input.
common.c / common.h	Shared constants (FIFO_PATH = /tmp/os_project_fifo, BUFFER_SIZE = 256) and helpers print_header(), print_subheader(), print_step(), pause_output().
ipc_session.c (core)	ipc_session_pipe(), ipc_session_fifo() and ipc_session_complete(); signal handling; session statistics; user message input.
ipc_threads.c	Message queue (mq_init, mq_push, mq_pop, mq_destroy), shared processing statistics, worker_thread() and run_workers().
co1_system_calls.c	System-call diagnostic: pipe, fork, read, write, close, wait, mkfifo and open with visible return values (menu option 7).
co4_memory.c	Memory diagnostic: data, stack and heap addresses, Copy-on-Write check and malloc error handling (option 5).
co5_files.c	File-system diagnostic: standard descriptors, FIFO creation, stat() output, reader/writer transfer and unlink (option 4).
co6_threads.c	Concurrency diagnostic: four threads incrementing a shared counter without and with a mutex (option 6).
co2_process_control.c, co3_ipc.c	Additional supporting modules for process control (fork, wait, exit status, optional zombie demonstration) and IPC concepts; they are compiled into the program.

5.5 Input and Output Design

Inputs

Table 5.2: Inputs

Input	Type / limit	Purpose
-------	--------------	---------

Input	Type / limit	Purpose
Menu choice	Integer 1 to 8	Selects a session or a diagnostic. Non-numeric input is rejected with a message and the menu is shown again.
Message text	Line of text, at most 255 characters	The data transferred from sender to receiver. Empty lines are ignored.
Ctrl+D (end of input)	End-of-file on the keyboard	Ends message entry and starts the closing sequence.
Ctrl+C (SIGINT)	Signal	Requests a controlled shutdown of the session.

Outputs

- Process information: parent PID and PPID, child PID, and the PID in every printed line.
- System-call trace: each call with its arguments and return value, for example `pipe()` followed by `read fd = 3` and `write fd = 4`.
- Message trace: sender line (`write(fd=4): "..."` (17 bytes)) and receiver line (Message 1: `"..."` (17 bytes)).
- FIFO metadata: file type, inode number and permissions from `stat()`.
- Worker-thread output and the two statistics boxes: processing statistics and session statistics.

Messages inside the pipe are separated by a newline character, so the receiver can rebuild the original messages from the byte stream. In FIFO mode each `read()` result is treated as one message. Console output uses fixed prefixes such as `[Parent PID=...]`, `[Child PID=...]`, `[Writer Parent ...]`, `[Reader Child ...]` and `[Thread n]`, which allow the actions of each process and thread to be told apart.

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

The program is implemented as ten C source files that are compiled into a single executable. The menu in `main.c` is the entry point, and each menu option calls one module. Table 6.1 summarises the implementation status of every module and the menu option through which it is reached.

Table 6.1: Menu options and modules

Menu option	Function called	Source file	What it demonstrates
1	<code>ipc_session_pipe()</code>	<code>ipc_session.c</code>	8–11
2	<code>ipc_session_fifo()</code>	<code>ipc_session.c</code>	12–15
3	<code>ipc_session_complete()</code>	<code>ipc_session.c</code>	16–17
4	<code>co5_demo()</code>	<code>co5_files.c</code>	18–20
5	<code>co4_demo()</code>	<code>co4_memory.c</code>	21–26
6	<code>co6_demo()</code>	<code>co6_threads.c</code>	27–38
7	<code>co1_demo()</code>	<code>co1_system_calls.c</code>	39–47
8	<code>return from main()</code>	<code>main.c</code>	48–50

Division of work among team members

The implementation work was divided so that each member owns one clearly defined part of the system.

Table 6.2: Individual contributions

Roll number	Student name	Individual responsibility
2520090181	Surapuraju Himasree	Design and implementation of anonymous pipe communication using <code>pipe()</code> , <code>fork()</code> , <code>read()</code> and <code>write()</code> .
2520080063	Mallula Lalitha	Design and implementation of named pipe (FIFO) communication using <code>mkfifo()</code> , <code>open()</code> , <code>read()</code> and <code>write()</code> .
2520090059	Sravva Peri	Integration and testing of both IPC mechanisms, process synchronization, error handling, output analysis and documentation.

Implementation of the shared infrastructure

Three pieces of infrastructure are shared by both IPC modes. The first is the **session statistics** structure, which holds four counters and a mutex. The second is the **signal-handling** code, in which the handlers set a `volatile sig_atomic_t` flag and use `write()` to print a short message, because `printf()` is not `async-signal-safe`. The third is the **message input** routine, which allocates a 256-byte buffer with `malloc()`, reads a line with `fgets()` and removes the

trailing newline.

6.2 Algorithm / Pseudocode

Algorithm 6.1: Anonymous pipe session

```
ALGORITHM PipeSession
  setup signal handlers; init statistics mutex
  pipe(pipefd)           // data channel
  pipe(stats_pipe)       // child -> parent statistics
  pid <- fork()
  if pid == 0:           // RECEIVER (child)
    close(pipefd[1]); close(stats_pipe[0])
    kill(getppid(), SIGUSR1) // tell parent: ready
    repeat
      n <- read(pipefd[0], &ch, 1)
      if n == 0: break           // EOF: writer closed
      if ch == '\n': queue.push(line); count++ ; line <- empty
      else append ch to line
    close(pipefd[0])
    run_workers(queue, 3 threads) // process messages
    write(stats_pipe[1], count); _exit(0)
  else:                 // SENDER (parent)
    close(pipefd[0]); close(stats_pipe[1])
    wait until SIGUSR1 is received
    while user enters messages:
      write(pipefd[1], msg); write(pipefd[1], "\n")
    close(pipefd[1]) // generates EOF for the child
    waitpid(pid); read(stats_pipe[0]); print statistics
END
```

Algorithm 6.2: Named FIFO session

```
ALGORITHM FifoSession
  unlink(FIFO_PATH); mkfifo(FIFO_PATH, 0666); stat(FIFO_PATH)
  block SIGUSR1 with sigprocmask()
  pid <- fork()
  if pid == 0:           // READER (child)
    fd <- open(FIFO_PATH, O_RDONLY) // blocks until writer opens
    kill(getppid(), SIGUSR1)
    repeat
      n <- read(fd, buf, 255)
      if n == 0: break           // EOF
      queue.push(buf, n)
    close(fd); run_workers(queue, 3); _exit(0)
  else:                 // WRITER (parent)
    fd <- open(FIFO_PATH, O_WRONLY) // blocks until reader opens
    sigsuspend() until SIGUSR1 arrives
    while user enters messages: write(fd, msg)
    close(fd); waitpid(pid); unlink(FIFO_PATH)
END
```

Algorithm 6.3: Worker thread

```
ALGORITHM WorkerThread(queue, stats)
  while queue.pop(&msg) succeeds: // pop is done under the queue mutex
    print "Processing message: msg"
    lock(stats.mutex)
    stats.processed++ ; stats.bytes += length(msg)
    unlock(stats.mutex)
  print "No more messages. Exiting."
END
```

6.3 Core C / C++ Implementation

This section presents the most important parts of the source code. The complete code is available in the GitHub repository <https://github.com/lalitha-0890/OS-Project>.

(a) Session statistics and signal handling - ipc_session.c

Listing 6.1: Mutex-protected session statistics

```
typedef struct {
    pthread_mutex_t mutex;
    long messages_sent;
    long messages_received;
    long bytes_transferred;
    long errors;
} session_stats_t;

static session_stats_t stats;

static void stats_init(void)
{
    memset(&stats, 0, sizeof(stats));
    if (pthread_mutex_init(&stats.mutex, NULL) != 0) {
        perror("pthread_mutex_init");
        exit(1);
    }
}

static void stats_destroy(void)
{
    pthread_mutex_destroy(&stats.mutex);
}

static void stats_record_sent(long bytes)
{
    pthread_mutex_lock(&stats.mutex);
    stats.messages_sent++;
    stats.bytes_transferred += bytes;
    pthread_mutex_unlock(&stats.mutex);
}

static void stats_record_received(long bytes)
{
    pthread_mutex_lock(&stats.mutex);
    stats.messages_received++;
    stats.bytes_transferred += bytes;
    pthread_mutex_unlock(&stats.mutex);
}
```

Listing 6.2: Async-signal-safe handlers installed with sigaction()

```
static volatile sig_atomic_t sigusr1_flag = 0;
static volatile sig_atomic_t sigint_flag = 0;

static void sigusr1_handler(int sig)
{
    (void)sig;
    sigusr1_flag = 1;
    const char *msg = " [Signal] SIGUSR1 received\n";
    write(STDOUT_FILENO, msg, strlen(msg));
}

static void sigint_handler(int sig)
{
    (void)sig;
    sigint_flag = 1;
    const char *msg = "\n [Signal] SIGINT received\n";
    write(STDOUT_FILENO, msg, strlen(msg));
}

static void setup_signal_handlers(void)
{
    struct sigaction sa_usr1, sa_int;

    memset(&sa_usr1, 0, sizeof(sa_usr1));
```

```

sa_usr1.sa_handler = sigusr1_handler;
sigemptyset(&sa_usr1.sa_mask);
sa_usr1.sa_flags = 0;
if (sigaction(SIGUSR1, &sa_usr1, NULL) == -1) {
    perror("sigaction SIGUSR1");
}

memset(&sa_int, 0, sizeof(sa_int));
sa_int.sa_handler = sigint_handler;
sigemptyset(&sa_int.sa_mask);
sa_int.sa_flags = 0;
if (sigaction(SIGINT, &sa_int, NULL) == -1) {
    perror("sigaction SIGINT");
}
}

```

(b) Anonymous pipe session - ipc_session.c

Listing 6.3: Creating the pipe and forking the receiver

```

/* Create pipe */
int pipefd[2];
printf("\n pipe()\n");
if (pipe(pipefd) == -1) {
    perror("pipe");
    stats_record_error();
    return;
}
printf("  read fd = %d\n", pipefd[0]);
printf("  write fd = %d\n", pipefd[1]);

/* Create stats pipe */
int stats_pipe[2];
if (pipe(stats_pipe) == -1) {
    perror("pipe (stats)");
    close(pipefd[0]);
    close(pipefd[1]);
    stats_record_error();
    return;
}

/* Copy-on-Write */
int cow_var = 100;
printf("\n Before fork: cow_var = %d at %p\n", cow_var, (void *)&cow_var);

/* Fork */
printf("\n fork()\n");
pid_t pid = fork();

if (pid == -1) {
    perror("fork");
    close(pipefd[0]);
    close(pipefd[1]);
    close(stats_pipe[0]);
    close(stats_pipe[1]);
    stats_record_error();
    return;
}

```

Listing 6.4: Receiver loop - rebuilding messages from the byte stream

```

size_t line_pos = 0;

while (1) {
    char ch;
    ssize_t n = read(pipefd[0], &ch, 1);
    if (n == -1) {
        perror("read");
        stats_record_error();
        break;
    }
}

```

```

    if (n == 0) {
        printf(" [Child PID=%d] Pipe EOF\n", getpid());
        break;
    }
    if (ch == '\n') {
        line[line_pos] = '\0';
        if (line_pos > 0) {
            msg_count++;
            printf(" [Child PID=%d] Message %d: \"%s\" (%zu bytes)\n",
                getpid(), msg_count, line, line_pos);
            mq_push(&mq, line, line_pos);
            stats_record_received(line_pos);
        }
        line_pos = 0;
    } else {
        if (line_pos < BUFFER_SIZE - 1) {
            line[line_pos++] = ch;
        }
    }
}

```

Listing 6.5: Sender - waiting for SIGUSR1 and writing messages

```

close(pipefd[0]);
printf(" [Parent PID=%d] close(fd=%d)\n", getpid(), pipefd[0]);

close(stats_pipe[1]);

printf(" [Parent PID=%d] Waiting for SIGUSR1...\n", getpid());
while (!sigusr1_flag && !sigint_flag) {
    pause();
}
printf(" [Parent PID=%d] SIGUSR1 received\n", getpid());

printf("\n [Parent PID=%d] Enter messages (Ctrl+D to finish):\n", getpid());

int msg_count = 0;
while (!sigint_flag) {
    char *msg = read_user_message();
    if (msg == NULL) {
        break;
    }

    if (strlen(msg) == 0) {
        free(msg);
        continue;
    }

    msg_count++;
    printf(" [Parent PID=%d] write(fd=%d): \"%s\" (%zu bytes)\n",
        getpid(), pipefd[1], msg, strlen(msg));

    ssize_t written = write(pipefd[1], msg, strlen(msg));
    if (written == -1) {
        perror("write");
        stats_record_error();
    } else {
        stats_record_sent(written);
    }
}

```

Listing 6.6: Closing the write end and reaping the child

```

printf("\n [Parent PID=%d] close(fd=%d)\n", getpid(), pipefd[1]);
close(pipefd[1]);

printf(" [Parent PID=%d] waitpid(%d)\n", getpid(), pid);
int status;
pid_t result = waitpid(pid, &status, 0);
if (result == -1) {
    perror("waitpid");
}

```

```

        stats_record_error();
    } else {
        printf("  [Parent PID=%d] Child %d terminated, status %d\n",
               getpid(), result, WEXITSTATUS(status));
    }
}

```

(c) Named FIFO session - ipc_session.c

Listing 6.7: Creating the FIFO, reading its metadata and blocking SIGUSR1

```

/* Create FIFO */
unlink(FIFO_PATH);

printf("\n  mkfifo(\"%s\", 0666)\n", FIFO_PATH);
if (mkfifo(FIFO_PATH, 0666) == -1) {
    perror("mkfifo");
    stats_record_error();
    return;
}
printf("  FIFO created: %s\n", FIFO_PATH);

/* stat() the FIFO */
struct stat st;
if (stat(FIFO_PATH, &st) == -1) {
    perror("stat");
    unlink(FIFO_PATH);
    stats_record_error();
    return;
}

printf("  File type:   %s\n", S_ISFIFO(st.st_mode) ? "FIFO" : "Unknown");
printf("  Inode:        %ld\n", (long)st.st_ino);
printf("  Permissions: %o\n", st.st_mode & 0777);

/* Block SIGUSR1 before fork to prevent interruption of open() */
sigset_t block_mask, orig_mask;
sigemptyset(&block_mask);
sigaddset(&block_mask, SIGUSR1);
if (sigprocmask(SIG_BLOCK, &block_mask, &orig_mask) == -1) {
    perror("sigprocmask");
    unlink(FIFO_PATH);
    stats_record_error();
    return;
}

/* Fork */

```

Listing 6.8: FIFO reader - open, notify and read loop

```

int fd_read = open(FIFO_PATH, O_RDONLY);
if (fd_read == -1) {
    perror("open (read)");
    fflush(stdout);
    _exit(1);
}
printf("  [Reader Child PID=%d] fd = %d\n", getpid(), fd_read);

/* Send SIGUSR1 to parent only after reader is ready */
printf("  [Reader Child PID=%d] kill(parent, SIGUSR1)\n", getpid());
if (kill(getppid(), SIGUSR1) == -1) {
    perror("kill");
}

/* Read all messages from FIFO into a message queue */
MessageQueue mq;
mq_init(&mq);

char *recv_buf = malloc(BUFFER_SIZE);
if (recv_buf == NULL) {
    perror("malloc");
}

```

```

        close(fd_read);
        _exit(1);
    }

    int msg_count = 0;
    while (1) {
        ssize_t n = read(fd_read, recv_buf, BUFFER_SIZE - 1);
        if (n == -1) {
            if (errno == EINTR) continue;
            perror("read");
            stats_record_error();
            break;
        }
        if (n == 0) {
            printf(" [Reader Child PID=%d] FIFO EOF\n", getpid());
            break;
        }
        recv_buf[n] = '\0';
        msg_count++;
        printf(" [Reader Child PID=%d] Message %d: \"%s\" (%zd bytes)\n",
            getpid(), msg_count, recv_buf, n);
        mq_push(&mq, recv_buf, n);
        stats_record_received(n);
    }

```

Listing 6.9: FIFO writer - blocking open and sigsuspend()

```

int fd_write = open(FIFO_PATH, O_WRONLY);
if (fd_write == -1) {
    perror("open (write)");
    waitpid(pid, NULL, 0);
    unlink(FIFO_PATH);
    stats_record_error();
    return;
}
printf(" [Writer Parent PID=%d] fd = %d\n", getpid(), fd_write);

/* Now unblock SIGUSR1 and wait for child ready signal */
printf(" [Writer Parent PID=%d] Waiting for SIGUSR1...\n", getpid());
sigset_t wait_mask;
sigemptyset(&wait_mask);
while (!sigusr1_flag && !sigint_flag) {
    sigsuspend(&wait_mask);
}
printf(" [Writer Parent PID=%d] SIGUSR1 received\n", getpid());

```

(d) Message queue and worker threads - ipc_threads.c

Listing 6.10: Thread-safe message queue

```

void mq_push(MessageQueue *mq, const char *msg, size_t len)
{
    pthread_mutex_lock(&mq->mutex);

    if (mq->count >= mq->capacity) {
        mq->capacity *= 2;
        mq->messages = realloc(mq->messages, mq->capacity * sizeof(char *));
        if (mq->messages == NULL) {
            perror("realloc");
            pthread_mutex_unlock(&mq->mutex);
            exit(1);
        }
    }

    mq->messages[mq->count] = malloc(len + 1);
    if (mq->messages[mq->count] == NULL) {
        perror("malloc");
        pthread_mutex_unlock(&mq->mutex);
        exit(1);
    }
    memcpy(mq->messages[mq->count], msg, len);
}

```

```

mq->messages[mq->count][len] = '\0';
mq->count++;

pthread_mutex_unlock(&mq->mutex);
}

int mq_pop(MessageQueue *mq, char **msg, size_t *len)
{
    pthread_mutex_lock(&mq->mutex);

    if (mq->next_index >= mq->count) {
        pthread_mutex_unlock(&mq->mutex);
        return 0;
    }

    *msg = mq->messages[mq->next_index];
    *len = strlen(*msg);
    mq->next_index++;

    pthread_mutex_unlock(&mq->mutex);
    return 1;
}

```

Listing 6.11: Worker thread

```

void *worker_thread(void *arg)
{
    worker_arg_t *warg = (worker_arg_t *)arg;
    char *msg = NULL;
    size_t len = 0;

    while (mq_pop(warg->mq, &msg, &len)) {
        printf(" [Thread %d] Processing message: \"%s\" (%zu bytes)\n",
            warg->thread_id, msg, len);
        ipc_stats_record(warg->stats, len);
    }

    printf(" [Thread %d] No more messages. Exiting.\n", warg->thread_id);

    free(warg);
    return NULL;
}

```

Listing 6.12: Creating and joining the worker threads

```

void run_workers(MessageQueue *mq, ipc_stats_t *s, int num_threads)
{
    pthread_t threads[WORKER_THREADS];
    int i;

    printf("\n Starting %d worker threads...\n", num_threads);

    for (i = 0; i < num_threads; i++) {
        worker_arg_t *arg = malloc(sizeof(worker_arg_t));
        if (arg == NULL) {
            perror("malloc");
            return;
        }
        arg->mq = mq;
        arg->stats = s;
        arg->thread_id = i + 1;

        if (pthread_create(&threads[i], NULL, worker_thread, arg) != 0) {
            perror("pthread_create");
            free(arg);
            return;
        }
    }
}

```

```

    for (i = 0; i < num_threads; i++) {
        if (pthread_join(threads[i], NULL) != 0) {
            perror("pthread_join");
        }
    }

    printf(" pthread_join(): all worker threads completed\n");
    printf(" Mutex synchronization: PASSED\n");
}

```

6.4 Linux System Calls / APIs Used

Table 6.3: System calls and APIs used

Call	Prototype (simplified)	How it is used
pipe	int pipe(int fd[2])	Returns read end fd[0] and write end fd[1]; a second pipe returns the receiver statistics.
mkfifo	int mkfifo(const char *path, mode_t mode)	Creates /tmp/os_project_fifo with mode 0666 (reduced by the umask).
fork	pid_t fork(void)	Creates the receiver; returns 0 in the child.
open	int open(const char *path, int flags)	O_RDONLY in the reader and O_WRONLY in the writer; each blocks until the other arrives.
read	ssize_t read(int fd, void *buf, size_t n)	Returns bytes read, 0 at EOF and -1 on error; EINTR is handled in the FIFO reader.
write	ssize_t write(int fd, const void *buf, size_t n)	Sends the message bytes; also used inside signal handlers.
close	int close(int fd)	Releases unused descriptors; closing the write end creates EOF.
waitpid	pid_t waitpid(pid_t pid, int *st, int opt)	Waits for the child and extracts the exit status with WEXITSTATUS().
kill	int kill(pid_t pid, int sig)	Child sends SIGUSR1 to the parent.
sigaction	int sigaction(int sig, const struct sigaction *act, ...)	Installs the handlers for SIGUSR1 and SIGINT.
sigprocmask / sigsuspend	int sigprocmask(...), int sigsuspend(const sigset_t *)	Blocks SIGUSR1 during open() and then waits for it without a race.
stat	int stat(const char *path, struct stat *st)	Gives the inode (st_ino), type (S_ISFIFO) and permissions (st_mode).
unlink	int unlink(const char *path)	Deletes the FIFO entry during cleanup.
malloc / realloc / free	void *malloc(size_t n) ...	Message buffers, the message array and thread arguments.
pthread_create / pthread_join	int pthread_create(...), int pthread_join(...)	Start and wait for the three worker threads.
pthread_mutex_lock / unlock	int pthread_mutex_lock(pthread_mutex_t *)	Protect the queue and the statistics.

6.5 Process / Thread Implementation

Processes

Each IPC session creates exactly one child process with `fork()`. After the call, both processes continue from the same point and distinguish their roles by the return value. In pipe mode the parent is the sender and the child is the receiver; in FIFO mode the parent is the writer and the

child is the reader. The program prints `getpid()` and `getppid()` so that the relationship is visible: in the sample run the parent had PID 327 and the child printed PPID=327. The parent always calls `waitpid()` so that the child cannot remain a zombie, and the child leaves with `_exit(0)` after flushing its output, which prevents it from running the parent's cleanup code twice.

Threads

After the receiver has read all messages, it starts **three worker threads** (`WORKER_THREADS`) inside the child process. Every thread runs `worker_thread()`, repeatedly takes the next message from the shared queue and records its length in the shared statistics. Each thread receives its own heap-allocated argument structure containing the queue pointer, the statistics pointer and its thread number; the thread frees this structure before it returns. The creating code calls `pthread_join()` on all three threads, so the final statistics are printed only after all work is complete.

Threads are created after the receive loop and not during it. This keeps the byte-stream parsing in a single thread and uses the threads only for the processing stage, which avoids unnecessary interaction between reading and processing.

6.6 IPC / Synchronization Implementation

Pipe and FIFO synchronization

- **Blocking reads and writes:** the kernel suspends a reader while the channel is empty and a writer while it is full, so no busy waiting is needed.
- **End-of-file:** the parent closes the write end after the last message. When every write end is closed, the child's `read()` returns 0 and the receive loop ends. For this to work, the child must close its own copy of the write end, which the implementation does immediately after `fork()`.
- **Blocking open for FIFOs:** `open(O_RDONLY)` and `open(O_WRONLY)` wait for each other, so the reader and the writer meet before any data moves.
- **Message boundaries:** in pipe mode a newline is written after every message and the receiver splits the stream at newlines. In FIFO mode each `read()` is treated as one message.

Signal-based handshake

The child sends `SIGUSR1` to its parent when it is ready. The parent does not ask for input until the signal has arrived. In pipe mode the parent waits in a loop around `pause()`. In FIFO mode

the parent first blocks `SIGUSR1` with `sigprocmask()` before forking, because a signal arriving during `open()` could interrupt it; after the `open` completes the parent waits with `sigsuspend()`, which atomically unblocks the signal and sleeps, so a signal that arrives early cannot be lost. The handler only sets a flag and writes a short line with `write()`.

Mutual exclusion

Three structures are protected by a mutex: the session statistics, the message queue and the processing statistics. Every access follows the same pattern: lock, update, unlock. The functions `mq_push()` and `mq_pop()` hold the queue mutex during the whole operation, so two worker threads can never take the same message. The `realloc()` that grows the queue also runs while the mutex is held. In the concurrency diagnostic (option 6), four threads each add 1 to a shared counter 100,000 times, first without a lock and then with `pthread_mutex_lock()`; the mutex version always ends with exactly 400,000.

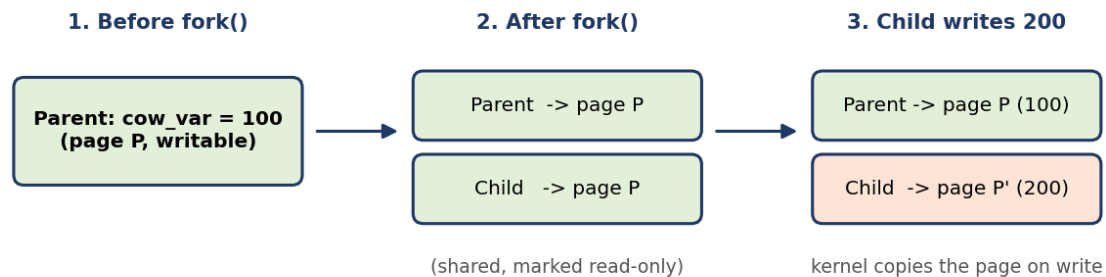
Return of statistics through a second pipe

The receiver's received-message count lives in the child's memory and is not visible to the parent. The parent therefore creates a second pipe before `fork()`; the child writes one `long` value into it just before exiting, and the parent reads it after `waitpid()`. This turns the project's own IPC mechanism into the solution to a real data-sharing problem.

6.7 Memory Management Implementation

- **Heap buffers:** each user message is read into a 256-byte buffer obtained from `malloc()` and released with `free()` after it has been written. The receiver uses a line buffer and a receive buffer that are freed after the read loop.
- **Dynamic message queue:** the queue stores `char *` pointers in an array that starts with a capacity of 16 and doubles with `realloc()` when full. Every message is copied into its own `malloc()` block and released in `mq_destroy()`.
- **Error handling:** every `malloc()` result is checked; on failure the code prints a message with `perror()` and releases what it already holds. The memory diagnostic requests an impossible allocation and shows `malloc() returned NULL, errno = 12 (Cannot allocate memory)`.
- **Copy-on-Write:** the pipe session sets `cow_var = 100` before `fork()`, the child changes it to 200, and the parent still prints 100. The memory diagnostic repeats the experiment for a global (data segment), a stack and a heap variable and prints "Copy-on-Write: VERIFIED".

- **Address-space regions:** the diagnostic prints the addresses of a global variable (data), a local variable (stack) and a `malloc()` block (heap). The child prints the same virtual addresses with different values, which shows that the two processes have separate address spaces that use identical virtual addresses.



Result: parent still prints 100, child prints 200 -> "Copy-on-Write: VERIFIED"

Figure 6.1: Copy-on-Write after fork()

6.8 Optional User Interface Implementation

The project uses a console menu as its user interface; no graphical interface is required. The function `print_menu()` in `main.c` lists the eight options and `main()` reads the choice with `scanf()`. After reading the number the program discards the rest of the line so that the later `fgets()` call for message text works correctly. Non-numeric input produces the message "Invalid input. Please enter a number." and an out-of-range number produces "Invalid option. Please choose 1-8." At end of input the program exits safely. The menu is displayed again after every action, so several sessions can be run one after the other.

Main menu

```
=====
LINUX IPC MONITORING & MESSAGE TRANSFER
=====

1. Start integrated IPC session - anonymous pipe
2. Start integrated IPC session - named FIFO
3. Run complete end-to-end demonstration
4. Inspect IPC resources and file descriptors
5. Run memory/Copy-on-Write diagnostic
6. Run concurrency and mutex diagnostic
7. Run system-call verification instructions
8. Exit safely

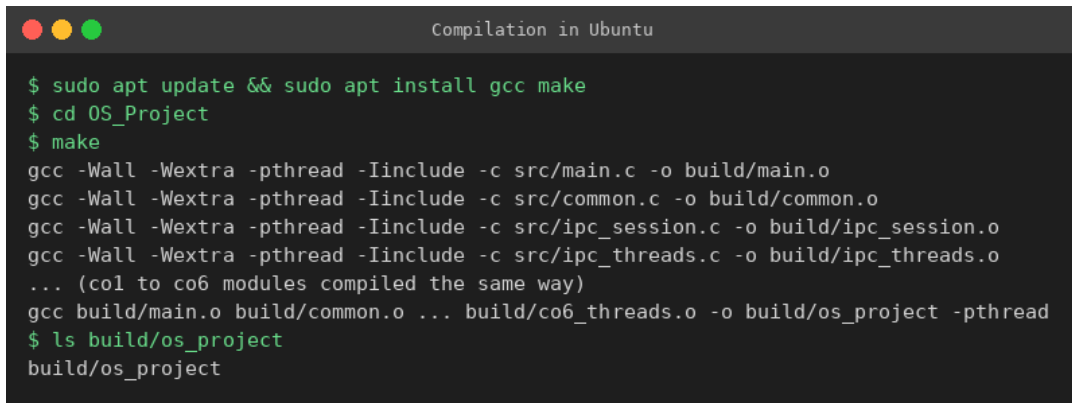
Choose an option:
```

CHAPTER -7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

The project is compiled with GNU Make. The Makefile compiles every .c file in src/ into an object file in build/ using gcc -Wall -Wextra -pthread -Iinclude and then links all object files into build/os_project. The build produces no errors.

```
cd OS_Project
make          # build -> build/os_project
make clean    # remove the build/ directory
make run      # build and start the program
```



```
$ sudo apt update && sudo apt install gcc make
$ cd OS_Project
$ make
gcc -Wall -Wextra -pthread -Iinclude -c src/main.c -o build/main.o
gcc -Wall -Wextra -pthread -Iinclude -c src/common.c -o build/common.o
gcc -Wall -Wextra -pthread -Iinclude -c src/ipc_session.c -o build/ipc_session.o
gcc -Wall -Wextra -pthread -Iinclude -c src/ipc_threads.c -o build/ipc_threads.o
... (col to co6 modules compiled the same way)
gcc build/main.o build/common.o ... build/co6_threads.o -o build/os_project -pthread
$ ls build/os_project
build/os_project
```

Figure 7.1: Compilation of the project with make

7.2 Execution Commands

Table 7.1: Execution commands

Purpose	Command
Run the program	./build/os_project
Anonymous pipe session	Choose option 1, type messages, finish with Ctrl+D
Named FIFO session	Choose option 2, type messages, finish with Ctrl+D
Full demonstration	Choose option 3
See the FIFO from a second terminal	ls -l /tmp/os_project_fifo and stat /tmp/os_project_fifo
Trace system calls	strace -f -e trace=pipe,pipe2,fork,clone,read,write,openat,close,mkfifo,wait4 ./build/os_project
Check for zombie processes	ps aux grep Z
Check for memory leaks	valgrind --leak-check=full ./build/os_project

7.3 Sample Inputs and Outputs

Table 7.2: Sample inputs and outputs

Session	Messages entered	Observed result
Anonymous pipe (option 1)	"Hello from parent" (17 bytes) "Second message" (14 bytes)	Both messages received by the child; messages sent 2, received 2, bytes transferred 31, errors 0.

Session	Messages entered	Observed result
Named FIFO (option 2)	"Hello via FIFO" (14 bytes) "Second FIFO message" (19 bytes)	FIFO created with file type FIFO; both messages received by the reader; 33 bytes processed; FIFO removed.
File-system diagnostic (option 4)	None	18 bytes "Data through FIFO!" read by the child; FIFO shown with 0 bytes size and 1 link, then removed.
Memory diagnostic (option 5)	None	Child values 200, parent values 100: "Copy-on-Write: VERIFIED"; malloc failure reports errno 12.
Concurrency diagnostic (option 6)	Press Enter at the pause	Expected 400000, actual 400000 with the mutex: PASSED.

The sample output of the pipe session, trimmed to the important lines, is shown below. The two messages typed by the user appear first in the sender lines and then in the receiver lines with the same text and byte counts.

Sample output - anonymous pipe session

```
pipe()
read fd = 3
write fd = 4

fork()
[Parent PID=327] fork() -> child PID 328
[Child PID=328] PPID=327
[Child PID=328] cow_var = 200 (modified)
[Parent PID=327] cow_var = 100 (unchanged)
[Child PID=328] kill(parent, SIGUSR1)
[Parent PID=327] SIGUSR1 received

[Parent PID=327] write(fd=4): "Hello from parent" (17 bytes)
[Child PID=328] Message 1: "Hello from parent" (17 bytes)
[Parent PID=327] write(fd=4): "Second message" (14 bytes)
[Child PID=328] Message 2: "Second message" (14 bytes)
[Parent PID=327] close(fd=4)
[Child PID=328] Pipe EOF
[Parent PID=327] Child 328 terminated, status 0

| Messages sent:      2
| Messages received:  2
| Bytes transferred:  31
| Errors:             0
```

7.4 Screenshots of Execution

The figures in this section show the terminal output of the application for each menu option. The process identifiers, addresses and inode numbers differ from run to run.

```
$ ./build/os_project (option 1)

$ ./build/os_project
Choose an option: 1
=====
IPC SESSION - ANONYMOUS PIPE
=====

Parent PID : 327
Parent PPID: 326

pipe()
read fd = 3
write fd = 4

Before fork: cow_var = 100 at 0x7ffc0bef4ed8

fork()
[Parent PID=327] fork() -> child PID 328
[Parent PID=327] cow_var = 100 (unchanged)
[Parent PID=327] close(fd=3)
[Parent PID=327] Waiting for SIGUSR1...
[Child PID=328] PPID=327
[Child PID=328] cow_var = 200 (modified)
[Child PID=328] close(fd=4)
[Child PID=328] kill(parent, SIGUSR1)
[Child PID=328] read(fd=3)
[Signal] SIGUSR1 received
[Parent PID=327] SIGUSR1 received

[Parent PID=327] Enter messages (Ctrl+D to finish):

Enter a message to send (max 255 chars): Hello from parent
[Parent PID=327] write(fd=4): "Hello from parent" (17 bytes)

[Child PID=328] Message 1: "Hello from parent" (17 bytes)
Enter a message to send (max 255 chars): Second message
[Parent PID=327] write(fd=4): "Second message" (14 bytes)
```

Figure 7.2: Anonymous pipe session (option 1) - process creation and message transfer

```
$ ./build/os_project (option 1, continued)

[Child PID=328] Message 2: "Second message" (14 bytes)
Enter a message to send (max 255 chars):
[Parent PID=327] close(fd=4)
[Parent PID=327] waitpid(328)
[Child PID=328] Pipe EOF
[Child PID=328] close(fd=3)

Starting 3 worker threads...
[Thread 1] Processing message: "Hello from parent" (17 bytes)
[Thread 1] Processing message: "Second message" (14 bytes)
[Thread 1] No more messages. Exiting.
[Thread 2] No more messages. Exiting.
[Thread 3] No more messages. Exiting.
pthread_join(): all worker threads completed
Mutex synchronization: PASSED

+-----+
| IPC PROCESSING STATISTICS |
+-----+
| Messages received: 2
| Messages processed: 2
| Bytes processed: 31
+-----+
[Parent PID=327] Child 328 terminated, status 0
[Parent PID=327] Child reported 2 messages received
[Parent PID=327] Session complete.

+-----+
| SESSION STATISTICS |
+-----+
| Messages sent: 2
| Messages received: 2
| Bytes transferred: 31
| Errors: 0
+-----+
```

Figure 7.3: Anonymous pipe session - worker threads and statistics

```
$ ./build/os_project (option 2)

Choose an option: 2
=====
IPC SESSION - NAMED PIPE (FIFO)
=====

Parent PID : 332
Parent PPID: 326

mkfifo("/tmp/os_project_fifo", 0666)
FIFO created: /tmp/os_project_fifo
File type:  FIFO
Inode:      2801665
Permissions: 644

fork()
[Writer Parent PID=332] fork() -> child PID 333
[Writer Parent PID=332] open("/tmp/os_project_fifo", O_WRONLY)
[Reader Child PID=333] PPID=332
[Reader Child PID=333] open("/tmp/os_project_fifo", O_RDONLY)
[Reader Child PID=333] fd = 3
[Reader Child PID=333] kill(parent, SIGUSR1)
[Writer Parent PID=332] fd = 3
[Writer Parent PID=332] Waiting for SIGUSR1...
[Signal] SIGUSR1 received
[Writer Parent PID=332] SIGUSR1 received

[Writer Parent PID=332] Enter messages (Ctrl+D to finish):

Enter a message to send (max 255 chars): Hello via FIFO
[Writer Parent PID=332] write(fd=3): "Hello via FIFO" (14 bytes)

[Reader Child PID=333] Message 1: "Hello via FIFO" (14 bytes)
Enter a message to send (max 255 chars): Second FIFO message
[Writer Parent PID=332] write(fd=3): "Second FIFO message" (19 bytes)
[Reader Child PID=333] Message 2: "Second FIFO message" (19 bytes)
```

Figure 7.4: Named FIFO session (option 2) - FIFO creation and message transfer

```
$ ./build/os_project (option 2, continued)

Enter a message to send (max 255 chars):
[Writer Parent PID=332] close(3)
[Reader Child PID=333] FIFO EOF
[Reader Child PID=333] close(3)

Starting 3 worker threads...
[Writer Parent PID=332] waitpid(333)
[Thread 1] Processing message: "Hello via FIFO" (14 bytes)
[Thread 1] Processing message: "Second FIFO message" (19 bytes)
[Thread 1] No more messages. Exiting.
[Thread 2] No more messages. Exiting.
[Thread 3] No more messages. Exiting.
pthread_join(): all worker threads completed
Mutex synchronization: PASSED

+-----+
| IPC PROCESSING STATISTICS |
+-----+
| Messages received: 2
| Messages processed: 2
| Bytes processed: 33
+-----+
[Writer Parent PID=332] Child 333 terminated, status 0
[Writer Parent PID=332] unlink("/tmp/os_project_fifo")
[Writer Parent PID=332] FIFO removed.
[Writer Parent PID=332] Session complete.

+-----+
| SESSION STATISTICS |
+-----+
| Messages sent: 2
| Messages received: 0
| Bytes transferred: 33
| Errors: 0
+-----+
```

Figure 7.5: Named FIFO session - worker threads, cleanup and statistics

```
Option 4 - File descriptors and FIFO inspection

Choose an option: 4
=====
CO-5: FILESYSTEM DIAGNOSTIC
=====

fd 0 = stdin, fd 1 = stdout, fd 2 = stderr

Calling mkfifo("/tmp/os_project_fifo", 0666)...
FIFO created.

Path:      /tmp/os_project_fifo
File type: FIFO (named pipe)
Permissions: rw-r--r--  Inode:      2801665
Size:      0 bytes
Links:      1

Forking: parent = writer, child = reader...
[Writer Parent PID=337] Opening FIFO...
[Reader Child PID=338] Opening FIFO...
[Reader Child PID=338] fd = 3
[Writer Parent PID=337] fd = 3
[Writer Parent PID=337] write(fd=3, "Data through FIFO!", 18)
[Writer Parent PID=337] File closed.
[Reader Child PID=338] read() = 18 bytes: "Data through FIFO!"
[Reader Child PID=338] File closed.
[Writer Parent PID=337] Child terminated, status = 0

Calling unlink("/tmp/os_project_fifo")...
FIFO removed successfully.
```

Figure 7.6: File-descriptor and FIFO inspection (option 4)

```
Option 5 - Memory / Copy-on-Write diagnostic

Choose an option: 5
=====
CO-4: MEMORY DIAGNOSTIC
=====

global_value (data) = 100 at 0x55b03ac39010
stack_var   (stack) = 100 at 0x7fff316f48cc
heap_var    (heap)  = 100 at 0x55b0744b12c0

Calling fork()...
[Parent PID=339]
[Child  PID=340]
[Child] global_value = 200 at 0x55b03ac39010
[Child] *heap_var    = 200 at 0x55b0744b12c0
[Child] stack_var    = 200 at 0x7fff316f48cc
[Child] Memory freed.
[Parent] global_value = 100 at 0x55b03ac39010
[Parent] *heap_var    = 100 at 0x55b0744b12c0
[Parent] stack_var    = 100 at 0x7fff316f48cc

Copy-on-Write: VERIFIED

Testing malloc error handling...
malloc() returned NULL, errno = 12 (Cannot allocate memory)
[Parent] Memory freed.
```

Figure 7.7: Memory and Copy-on-Write diagnostic (option 5)

```
Option 6 - Concurrency and mutex diagnostic

Choose an option: 6
=====
CO-6: CONCURRENCY DIAGNOSTIC
=====

Threads: 4
Increments per thread: 100000

Without mutex:

Creating 4 threads (100000 increments each)...
[Thread 0] Done
[Thread 1] Done
[Thread 2] Done
[Thread 3] Done
Expected: 400000
Actual: 400000
Result: No race detected this run

[Press Enter to continue...]
With mutex:

Creating 4 threads with mutex...
[Thread 1] Done
[Thread 2] Done
[Thread 3] Done
[Thread 0] Done
Expected: 400000
Actual: 400000
Result: PASSED
```

Figure 7.8: Concurrency and mutex diagnostic (option 6)

```
Option 7 - System-call verification

Choose an option: 7
=====
CO-1: SYSTEM CALLS
=====

pipe()
read fd = 3
write fd = 4

fork()
[Parent PID=350] write(fd=4) -> 29 bytes: "Hello via pipe() system call!"
[Parent PID=350] close(fd=4) -> success
[Child PID=351] read(fd=3) -> 29 bytes: "Hello via pipe() system call!"
[Child PID=351] close(fd=3) -> success
[Parent PID=350] wait() -> child 351 exited, status 0

mkfifo("/tmp/os_project_fifo", 0666)
FIFO created at: /tmp/os_project_fifo

open("/tmp/os_project_fifo", O_RDONLY)
fd = 3
close(3) -> success

open("/tmp/os_project_fifo", O_WRONLY)
```

Figure 7.9: System-call verification (option 7)

In Figure 7.9 the last step tries to open the FIFO for writing in non-blocking mode while no

reader exists. The kernel refuses with the error "No such device or address" (ENXIO). This is the expected behaviour of a FIFO, and it shows that a writer cannot open a FIFO until a reader is present.

7.5 Demonstration of OS Concepts

Table 7.3: OS concepts demonstrated

OS concept	Evidence in the output
System calls / OS as a service layer	Each call is printed with its result: pipe() returns descriptors 3 and 4; fork() returns the child PID; read() and write() return byte counts.
Process creation and PIDs	Parent PID 327 creates child PID 328, and the child reports PPID=327.
Process synchronization	The parent waits for SIGUSR1, then sends messages; waitpid() returns the child with status 0, so no zombie is left.
Anonymous pipe	Descriptors 3 and 4 exist only inside the processes; the child receives EOF after the parent closes fd 4.
Named pipe (FIFO)	The FIFO has a path name, an inode number and the file type FIFO; it is removed by unlink().
Signals	"[Signal] SIGUSR1 received" is printed by the handler; the parent resumes after it.
Memory management	Heap buffers are allocated and freed; the global, stack and heap variables stay at 100 in the parent while the child changes them to 200 (Copy-on-Write).
File descriptors and file system	Descriptors 0, 1 and 2 are standard streams; new descriptors start at 3; stat() reports inode, size 0 bytes and link count 1.
Threads and mutex	Three worker threads share the message queue; the processing statistics match the number of messages and bytes sent; the mutex counter reaches 400000.

CHAPTER -8 CONCLUSION

8.1 Summary of the Project

This project studied and implemented Inter-Process Communication in Linux using anonymous pipes and named pipes (FIFOs). The result is a menu-driven C application, the Linux IPC Monitoring and Message Transfer System, in which a sender process transfers real, user-typed messages to a receiver process through a kernel pipe or FIFO. The program creates the processes with `fork()`, creates the channels with `pipe()` and `mkfifo()`, moves data with `read()` and `write()`, synchronizes the processes with signals, end-of-file and `waitpid()`, processes the received messages with worker threads and finally releases every resource.

Around the central message-transfer workflow, the application also demonstrates system calls, process identifiers, file descriptors and inodes, Copy-on-Write memory behaviour and the difference between an unprotected and a mutex-protected shared counter. Every step prints its system call and result, so the behaviour of the Linux kernel can be observed directly and confirmed with external tools such as `strace`, `ls -l` and `stat`.

8.2 Objectives Achieved

Table 8.1: Objectives and their status

Objective	Status	How it was achieved
IPC between related processes using anonymous pipes	Achieved	Option 1: parent sender and child receiver communicate through <code>pipe()</code> ; unused ends are closed and EOF terminates the transfer.
IPC between unrelated processes using named pipes	Achieved	Option 2: separate writer and reader open <code>/tmp/os_project_fifo</code> created by <code>mkfifo()</code> ; the two sides rendezvous at <code>open()</code> .
Reliable transfer, synchronization and coordination	Achieved	SIGUSR1 handshake, blocking open, EOF detection, <code>waitpid()</code> and mutex-protected statistics; sent and received counts and byte totals match in the sample runs.
Apply <code>pipe()</code> , <code>mkfifo()</code> , <code>fork()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code>	Achieved	All six calls are used with error checking and their results are printed.
Memory and Copy-on-Write visualization	Achieved	Options 1 and 5 show unchanged parent values after the child modifies its copy.
File-system connection of FIFOs	Achieved	<code>stat()</code> output shows inode, file type, permissions, size and link count.
Race condition and mutex synchronization	Achieved	Option 6 and the worker threads show the mutex-protected result of exactly 400000.
Controlled signal handling and clean resource management	Achieved	<code>sigaction()</code> handlers, descriptors closed, buffers freed, children reaped, FIFO unlinked.

8.3 Results Achieved

- The anonymous pipe session transferred two messages of 17 and 14 bytes (31 bytes in total) from the parent to the child with 0 errors, and the child reported both messages as received and processed.
- The named FIFO session transferred two messages of 14 and 19 bytes (33 bytes in total) between the writer and the reader processes. The FIFO was reported as type FIFO with its inode number and permissions 644 (the requested mode 0666 reduced by the default umask of 022), and it was removed afterwards.
- The three worker threads processed every queued message exactly once, and the processing statistics (messages processed and bytes processed) matched the transferred data.
- The memory diagnostic verified Copy-on-Write for data, stack and heap variables and reported errno 12 for an impossible allocation.
- The concurrency diagnostic produced 400000 increments with the mutex. Without the mutex the result may or may not be wrong, as it depends on thread timing; in the recorded run no loss was observed.
- No zombie processes remain after a session, because the parent always calls `waitpid()`.

Observations and limitations

- In FIFO mode the parent's final session statistics show "Messages received: 0", because the reader is a separate process and its counters are not sent back; the reader's own processing statistics show the correct values. Pipe mode does not have this effect because it returns the count through a second pipe. Adding the same mechanism to the FIFO session would remove the difference.
- In FIFO mode each `read()` is treated as one message and no delimiter is used, so two messages written very quickly could be read together. In this interactive program the sender waits for keyboard input between messages, so each message is read separately.
- The race-condition demonstration may not always show a wrong result, since the outcome depends on scheduling.
- Address values in the memory diagnostic vary from run to run because of address-space layout randomization (ASLR).
- The application is an educational demonstrator for a single machine and not a production message system.

Future scope

The project can be extended with bidirectional communication using two pipes, a multi-writer FIFO server, other IPC mechanisms such as POSIX message queues, shared memory with semaphores and sockets, and a performance comparison of the mechanisms for different message sizes.

8.4 Skills and Learning Outcomes

Table 8.2: Skills and learning outcomes

Area	Skills and knowledge gained
Operating system concepts	Process abstraction, user and kernel space, IPC, file descriptors, inodes, signals, Copy-on-Write and synchronization.
Linux system programming	Correct use of pipe(), mkfifo(), fork(), read(), write(), open(), close(), waitpid(), kill(), sigaction() and stat() with error handling.
Concurrent programming	Creating and joining POSIX threads, designing a thread-safe queue and using mutexes.
Resource management	Closing unused descriptors, freeing heap memory, reaping children and removing temporary files.
Tools	GCC, Make, strace, valgrind, Linux terminal commands, Git and GitHub.
Professional skills	Teamwork with clearly divided responsibilities, testing, output analysis and technical documentation.

CHAPTER -9 REFERENCES

9.1 Books

1. A. Silberschatz, P. B. Galvin and G. Gagne, **Operating System Concepts**, 10th ed., Wiley, 2018.
2. M. Kerrisk, **The Linux Programming Interface**, No Starch Press, 2010.
3. W. R. Stevens and S. A. Rago, **Advanced Programming in the UNIX Environment**, 3rd ed., Addison-Wesley, 2013.
4. A. S. Tanenbaum and H. Bos, **Modern Operating Systems**, 4th ed., Pearson, 2014.
5. R. Love, **Linux System Programming**, 2nd ed., O'Reilly Media, 2013.
6. B. W. Kernighan and D. M. Ritchie, **The C Programming Language**, 2nd ed., Prentice Hall, 1988.

9.2 Research Papers

1. D. M. Ritchie and K. Thompson, "The UNIX Time-Sharing System," **Communications of the ACM**, vol. 17, no. 7, pp. 365-375, 1974.
2. K. Thompson, "UNIX Implementation," **The Bell System Technical Journal**, vol. 57, no. 6, pp. 1931-1946, 1978.
3. E. W. Dijkstra, "Cooperating Sequential Processes," Technical report EWD-123, Technological University Eindhoven, 1965.
4. A. Baumann, J. Appavoo, O. Krieger and T. Roscoe, "A fork() in the road," in **Proc. 17th Workshop on Hot Topics in Operating Systems (HotOS)**, 2019.

9.3 Linux / GNU Documentation

1. Linux man-pages: pipe(2), pipe(7), fifo(7), mkfifo(3), fork(2), read(2), write(2), open(2), close(2), waitpid(2), kill(2), sigaction(2), signal(7), stat(2), unlink(2), pthreads(7) and malloc(3). Available at <https://man7.org/linux/man-pages/>.
2. The GNU C Library Reference Manual. Available at <https://www.gnu.org/software/libc/manual/>.
3. GCC, the GNU Compiler Collection - online documentation. Available at <https://gcc.gnu.org/onlinedocs/>.
4. GNU Make Manual. Available at <https://www.gnu.org/software/make/manual/>.

9.4 Online Resources

1. Project source code, GitHub repository: <https://github.com/lalitha-0890/OS-Project>.

2. Ubuntu documentation: <https://ubuntu.com/server/docs>.
3. Microsoft documentation, Windows Subsystem for Linux: <https://learn.microsoft.com/windows/wsl/>.
4. Project documentation files inside the repository: README.md, docs/architecture.md, docs/system_calls.md and docs/demonstration.md.